

Verlet Physics Sandbox

Leon Partington

Table Of Contents

<u>Table Of Contents</u>	<u>1</u>
<u>Analysis</u>	<u>3</u>
<u>Introduction</u>	<u>3</u>
<u>Target Audience</u>	<u>3</u>
<u>Research</u>	<u>3</u>
<u>Calculations</u>	<u>3</u>
<u>Verlet integration</u>	<u>4</u>
<u>Collision</u>	<u>4</u>
<u>Constraints</u>	<u>5</u>
<u>What Can Be Modeled With Verlet Integration?</u>	<u>6</u>
<u>Transverse Waves</u>	<u>6</u>
<u>Longitudinal Waves</u>	<u>6</u>
<u>Simple Harmonic Motion</u>	<u>6</u>
<u>Soft Body Objects</u>	<u>6</u>
<u>Engineering Shapes</u>	<u>6</u>
<u>Exploring The Specification of these concepts in various Subjects and Exams</u>	<u>7</u>
<u>Simple Harmonic Motion Within the A-Level AQA</u>	<u>7</u>
<u>Exploring Existing Systems</u>	<u>7</u>
<u>Phet Simulations</u>	<u>7</u>
<u>Hooke's Law Phet Simulation</u>	<u>7</u>
<u>My Physics Lab</u>	<u>8</u>
<u>Potential Platforms</u>	<u>8</u>
<u>Project Objectives</u>	<u>10</u>
<u>Design</u>	<u>12</u>
<u>Description</u>	<u>12</u>
<u>User Interface Design</u>	<u>13</u>
<u>Design Rules</u>	<u>14</u>
<u>Specific Design Choices</u>	<u>15</u>
<u>Tools</u>	<u>15</u>
<u>IPSO Chart</u>	<u>15</u>
<u>Flow Chart</u>	<u>17</u>
<u>Tools</u>	<u>18</u>
<u>Interact Mode</u>	<u>18</u>
<u>Edit Mode</u>	<u>19</u>
<u>Templates/ Pre-installed Meshes</u>	<u>20</u>
<u>Vector Graphics Library</u>	<u>23</u>
<u>Mesh</u>	<u>23</u>
<u>Particles</u>	<u>23</u>
<u>Sticks</u>	<u>23</u>
<u>Drawing</u>	<u>23</u>
<u>Physics Iteration</u>	<u>24</u>
<u>Colliders</u>	<u>24</u>

<u>Collision Resolution</u>	<u>24</u>
<u>Undo / Redo</u>	<u>24</u>
<u>MyStack</u>	<u>25</u>
<u>Factories</u>	<u>25</u>
<u>File Saving</u>	<u>25</u>
<u>Logger and Debug Console</u>	<u>26</u>
<u>Log Design</u>	<u>26</u>
<u>Debug Console</u>	<u>26</u>
<u>Environment Variables</u>	<u>27</u>
<u>Database</u>	<u>28</u>
<u>Entity Descriptions</u>	<u>28</u>
<u>Roles</u>	<u>28</u>
<u>Users</u>	<u>28</u>
<u>Structures</u>	<u>28</u>
<u>Entity Relationship Diagram</u>	<u>30</u>
<u>Testing</u>	<u>31</u>
<u>Fixes</u>	<u>38</u>
<u>Evaluation</u>	<u>39</u>
<u>Objectives</u>	<u>39</u>
<u>Conclusion</u>	<u>42</u>
<u>User Feedback</u>	<u>43</u>
<u>Improvements</u>	<u>44</u>
<u>Stability/performance</u>	<u>44</u>
<u>Usability</u>	<u>44</u>
<u>Database/Assignment improvement</u>	<u>44</u>
<u>Tearing/Stick Improvement</u>	<u>45</u>
<u>Notes.</u>	<u>45</u>

Analysis

Introduction

In Physics, students are taught lots of laws of the world through abstract concepts and badly drawn diagrams. One of the ways around this are E-Learning tools where teachers may play little animations or maybe even some simple simulations but none really suffice. The animations are usually stale and forgettable and the simulations restrictive and boring and for students to learn and really understand without endless drilling into their heads this data enjoyment is required. This is why I aim to create a physics sandbox that allows for the creation -through intuitive controls- particles connected to spring. While this concept is quite basic and does not leave much for students to explore as everything is simulated in 2D without any limitations many seemingly unrelated things can be simulated. For instance waves can be easily simulated by creating a line of close particles and springs and oscillation of a single particle.

Target Audience

In Physics and Mathematical Mechanics, Students encounter Physical interactions in both the Macro and micro scale e.g. “Elastic Tire” and a “Elastic Spring” Both of these interactions are both taught well and easily understood by students but how the connection is often overlooked. My Simulation would allow students to create physically accurate soft body objects out of Simple Springs. This simplicity allows a wide range of ages in various subjects to take advantage.

The program is not intended to be used as an E-Learning tool to teach new information but as a clarification how mathematics interact with the real world. This means that students must be able to easily interact, create and remove polygons in an entertaining fashion as to help build confidence.

Research

Calculations

From my research, I found that materials can be simulated using points and constraints. The most common algorithm for this is Verlet integration, which models each point (or particle) using equations based on the SUVAT formulas I learned in Physics. Normally, physics simulations use each particle's mass, position, and velocity, but Verlet integration uses mass, current position, and previous position instead. Velocity is then worked out automatically each tick.

This has a big advantage — if you change the position directly (for example, by locking a point in place or applying a constraint), the algorithm automatically handles the rest of the physics. While it's similar to basic Euler integration in small cases, Verlet integration is simpler and more stable when working with large numbers of particles interacting with each other, which means fewer bugs and smoother simulations. Furthermore, applying new

colliders is much easier to program due to only having to update the current position of a particle. While Verlet sounds like a perfect solution for all physics simulations it has a disadvantage of being unstable at low iterations with strong forces looking jittery at best and completely broken at worst and for this in my project I will need to have an interface to easily change the number of integrations and elasticity of the constraints.

Verlet integration

The actual equation is simple and at the most basic implementation all you need to store per particle is Current Position and previous position making have large amounts of particles. The equation is

$$\mathbf{x}_{n+1} = 2\mathbf{x}_n - \mathbf{x}_{n-1} + \mathbf{A}(\mathbf{x}_n) \Delta t^2$$

With X equaling position and N being the iteration and t being the time since last iteration. And A being the Acceleration calculated by Force over Mass and the force being whatever force you want to apply like gravity.

```
Class Particle
{
    Vec2 previousPos
    Vec2 currentPos
Particle(vec2 StartingPos)
{
    Vec2 previousPos = StartingPos;
    Vec2 currentPos = StartingPos;
}
Def Update(float Dtime)
{
    Vec2 accel = baseForce/mass;
    particle.previousPos = particle.currentPos
    particle.currentPos = 2*particle.currentPos -
particlePreviousPos + accel*(Dtime**2);
}
```

Collision

To apply a basic collision just change the currentPosition to the edge of the collider and the formula will do a basic resolution. This is because the velocity is calculated by subtracting the current position by the previous position and so if it reduces the velocity reduces enough that it stops. This type of collision is effective but does not properly collide elastically and thus does not bounce.

PseudoCode:

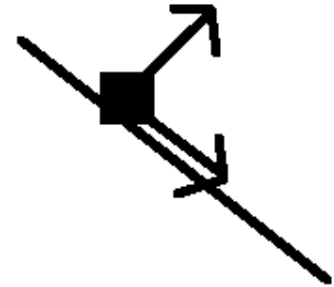
```
if(collides) {
    particle.currentPosition = colliderClampedPos;
```

}

You just need to do the collision detection which as its just point collision it shouldn't be too taxing on the computer.

This collision is inelastic and will not slide. The inelastic nature is easily solved in a soft body as the springs will bounce. But this is not adequate for a proper collision.

The next level of collision is a perfectly elastic collision where velocity is just flipped but this also isn't very realistic as no surface can create a perfectly elastic collision. The next solution is simple to apply a loss of energy in every collision but this can not be applied as a flat number as the level of grip and bounce is not equal such as ice: Not a partially elastic surface but has very low friction. This isn't as easy as just having a different energy loss coefficient for the x and y dimensions as not all collisions are aligned in on an axis like this. This where splitting components of the velocity into normal and tangential velocities is useful. These are found with :



Example of the different velocities

```
normalVel = Vector.dot(velocity,collisonNormal);
NormalComponent = collisonNormal* normalVel;
TangentComponent = velocity - NormalComponent;

NormalComponent *= NormalCoefficient;
TangentComponent *= TangentCoeffecient;

Velocity = NormalComponent + TangentComponent;
```

Constraints

To do constraints you must initialise a desired distance for the constraints to spring back too defined as d or as the magnitude of the distance between the two particles when the constraint is initialised. And when iterating the simulation calculate a difference ratio with `vec2 diffRatio = (desiredLength-lengthdiff)/lengthDiff;` then multiplying by a constant between 1 and 0 with this being the strength of the given constraint. Multiply this new factor with the initial length diff and add this to the position.

But once again I decided this was not enough even though it looked correct I wanted to apply more real world by using the spring constant equations Hooke's law: $F = -k\Delta x$ But to implement this you would have to store an accumulated force vector in each particle to be applied and zeroed at the end of every iteration

What Can Be Modeled With Verlet Integration?

From Testing with a combination of my basic prototype and online simulations I have found Multiple possible use cases for a simulation like this.



Transverse Waves

Transverse waves is a wave where the particles move perpendicular to the flow of energy for instance a oscillating taught rope. They can be simulated by connecting a line of particles and constraints with one fixed particle and one Oscillating particle while taught.



How a transverse could be made using particles and constraints



Longitudinal Waves

Longitudinal waves is a wave where the movement of particles is parallel to the flow of energy for instance how sound waves work. They could be simulated by creating a grid of particles with each corner fixed to a point and interacting with an individual particle to initialise a chain reaction.

A representation of longitudinal waves

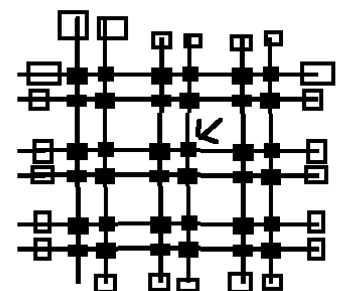


Simple Harmonic Motion

Simple Harmonic motion is the effect where a particle attached to a spring will oscillate and slowly be dampened as energy is lost in the elastic interaction. This can be simulated by simply fixing one particle and attaching a free moving particle with a spring.

Soft Body Objects

“Soft Body” is a general term for any polygon that is modelled as a collection of Constraints(Springs) and particles. They can be anything from a tire to jelly all the way to more rigid objects like bridges.



Engineering Shapes

While this is included within Soft body Objects this is an important concept to model. It includes the shapes and structures that can form strong rigid shapes that are used in all parts of engineering

Exploring The Specification of these concepts in various Subjects and Exams

Simple Harmonic Motion Within the A-Level AQA

As a Physics student you are expected to understand how a particle moves in relation to displacement, velocity and acceleration comparative to each other and time. This would need to be obvious in my simulation in my simulation

Simple harmonic motion

REVISED

$\text{acceleration } [a] = -kx$

where k is a constant and x is the displacement of the body from the fixed central point (O) at any time t .

Note that the maximum speed occurs at the centre of the oscillation while the maximum acceleration occurs at the two 'ends' of the oscillation (maximum displacement).

The maximum displacement of the body on either side of its central position is called the amplitude (A). The period of the motion (T) is the time it takes for the body to make one complete oscillation.

The frequency (f) is the number of complete oscillations per second, or $1/T$.

A body is undergoing simple harmonic motion (SHM) — Figure 6.2) if it has an acceleration that is:

- directed towards a fixed point, and
- proportional to the displacement of the body from that point.

Exploring Existing Systems

Phet Simulations

Hooke's Law Phet Simulation

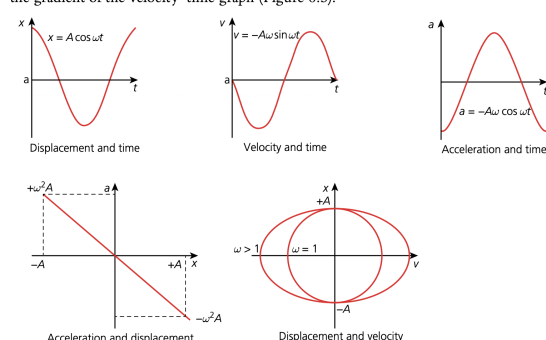
PhET is an interactive simulation platform developed by the University of Colorado Boulder, providing free science and mathematics simulations for educational use.

The Hooke's Law simulation allows a user to manipulate a spring system by

adjusting two parameters: the spring constant (measured in N/m) and the applied force (measured in N), with the resulting displacement shown visually on the canvas.

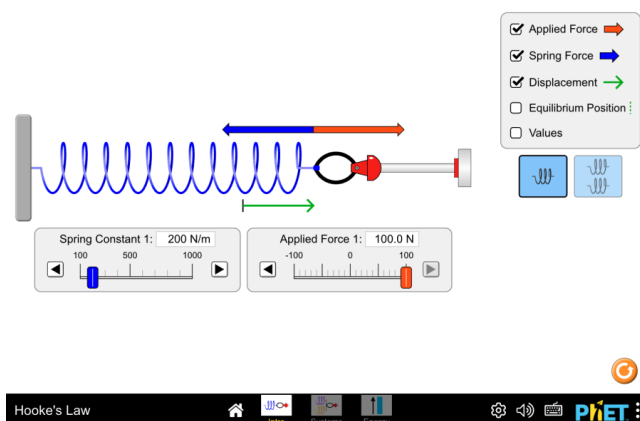
Simple harmonic motion graphs

Velocity is the gradient of the displacement–time graph. Acceleration is the gradient of the velocity–time graph (Figure 6.3).



This has very few parameters and is one of its strengths and limitations. The one dimension is very similar to the type of practical done in a lab. But if this simulation can be done in a lab with low cost materials that the majority of schools will have, its usefulness is limited. More variables could

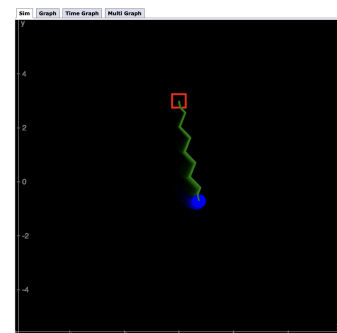
allow for a greater range of experiments that would be difficult to do in the lab, especially individually by every student.



There are more Simulations such as the [Pendulum Lab](#) and [Masses and Springs](#) which are useful in their own right but have similar limitations as hooke's law due to lack of parameters leading to impediments on students learning on how these equations occur in the real world.

[My Physics Lab](#)

My Physics Lab has a wide range of Simulations but each one is very limited in what you can do. Students are not able to create their own experiments and perform hypotheses as there is no freedom on what they can do



Potential Platforms

There are multiple ways to implement this kind of Simulation:

Platform	Pros	Cons
Browser Application	• Completely Cross Platform with No extra work for supporting almost any device • Easy Development on any Working environment e.g. Windows, Mac and linux • Updating the software for every user is automatic and should happen without the user even knowing • Easy Menu implementation using HTML	• More complicated Graphics are much more work compared to other platforms • Potentially slower due to limitations of the language • I am a beginner and not as comfortable as other platforms • Limitations of the language such as no operator overloads can lead to more

			programming time
Pygame , Pyarcade		• Lots of Experience • Fast Programming due to ease of use of the language and readability • Large ecosystem of libraries to reduce programming on solved irrelevant programming problems	• Language could be a bottleneck • Not easily Packaged in a user friendly executable
MonoGame		• Lots of Experience • Low language overhead • Most Recent Experience	• Menuing will either have to be done by hand or through an external library • Can Be difficult to switch between development environments especially from UNIX systems to windows
	DearImGuiNetMonogame	• Easy Implementation and integration with Monogame • Looks Modern • Has all benefits of Monogame but makes Menus easy	• Not on Nuget So integration requires adding the entire git repo as a git submodule • Low Experience
	WinFormsMonogame	• Menus Implemented through a GUI integration within VisualStudio	• Only works on windows making development difficult on my Laptop • Requires a Gui To change Windows leading to longer development times and more difficult debugging • Looks Dated • Low Experience
Unity		• Wide CrossPlatform Compatibility • Gui Interface	• Lots of code hidden behind a GUI • High Overhead GUI

After Weighing the Pros of Cons of these platforms I have chosen to move forward with Monogame Paired with ImGui. This Platform Combination was chosen due to my high experience with monogame and ease of use of ImGui.

Project Objectives

1. Mesh
 - 1.1. Mesh Representation
 - 1.1.1. Create a custom Class that Stores everything related that would make sense to be saved to do with the simulation
 - 1.1.2. Core Objects
 - 1.1.2.1. Stick
 - 1.1.2.1.1. Store Data in a class to do with the Stick or Constraint mainly the Natural Length and the particles that it is connected too
 - 1.1.2.2. Particle
 - 1.1.2.2.1. Store the Position of the particle, previous position and the accumulated force and mass in a class
 - 1.2. Core Mesh Functions
 - 1.2.1. Add Particle
 - 1.2.2. Add Stick Between Particles
 - 1.2.3. Remove Particle
 - 1.2.4. Remove Stick Between Particles
 - 1.2.5. Deep Copy for Multiple Mesh Instances
 - 1.3. Mesh Builder Helper Functions
 - 1.3.1. A function that creates a grid of particles and constraints and automatically pins the top two particles
 - 1.3.2. A function that easily allows the user to create polygons using only mouse inputs
 - 1.3.3. A line Generator allowing the creation of a line of particles with a definable density
 - 1.4. Mesh Drawing
 - 1.4.1. Add Functions on both the Stick And Particle classes for drawing
 - 1.4.2. The ability to colour particles and sticks based on interactions of the user and world on each individual particle
2. Physics
 - 2.1. Have a core Physics Iteration for the Particles
 - 2.2. Have a core Physics Iteration for the Sticks that applies the changes to the particles for the next particle physics iteration
3. User Interaction
 - 3.1. Mouse Interactable GUI
 - 3.1.1. Top Menu bar with openable windows and menus
 - 3.1.1.1. Should Be an Information Window that tells you how to use the application

- 3.1.1.2. Should be Menu for switching tool and tool mode
- 3.1.1.3. Should be a settings menu
- 3.1.1.4. Should be a menu for saving and loading Meshes onto the local device or the cloud through a database
- 3.2. Mouse Interactable Mesh
 - 3.2.1. The user should have multiple ways to interact with the Mesh named tools. These Tools should be split into two separate categories Interact and Edit to allow make finding different tools easier
 - 3.2.1.1. Tools and their modes should be changeable through Keyboard Inputs and GUI interactions. Each of the modifiers of the tools should also easily be modified
 - 3.2.1.1.1. Edit
 - 3.2.1.1.1.1. You should be able to build polygons through mouse inputs
 - 3.2.1.1.1.2. You should be able to build Lines of particles and sticks
 - 3.2.1.1.1.3. The user should be able delete and add particles
 - 3.2.1.1.1.4. The user should be able to add sticks between particles
 - 3.2.1.1.1.5. You should be able to place colliders
 - 3.2.1.1.1.6. You should be able to move colliders
 - 3.2.1.1.1.7. You should be able to delete colliders
 - 3.2.1.1.1.8. There should be an oscillating particle to create waves
 - 3.2.1.1.1.9. You should be able to create custom polygon colliders
 - 3.2.1.1.2.
 - 3.2.1.1.3. Interact
 - 3.2.1.1.3.1. Drag
 - 3.2.1.1.3.1.1. A drag that works by teleporting an area of particles to the mouse position
 - 3.2.1.1.3.1.2. A drag that works by applying a force to an area of particles to the mouse position. This force should be fully adjustable
 - 3.2.1.1.3.1.3. The drags area should be fully adjustable
 - 3.2.1.1.3.2. Cut Line
 - 3.2.1.1.3.2.1. You should be able to Cut all sticks in a line through a mouse input
 - 3.2.1.1.3.3. Select Particles
 - 3.2.1.1.3.3.1. You should be able to Select Particles and display relevant

- data about them such as position
and Accumulated force
 - 3.2.1.1.3.4. Pin
 - 3.2.1.1.3.4.1. You should be able to pin particles and turn off their physics
 - 3.2.1.1.3.5. Cut
 - 3.2.1.1.3.5.1. You should be able to delete all sticks in a radius
 - 3.2.1.1.3.6. Wind
 - 3.2.1.1.3.6.1. You should be able to apply wind to all particles like a temporary gravity
 - 3.2.1.1.3.7. Cursor collider
 - 3.2.1.1.3.7.1. You should be able to have a collider locked to the cursor to move particles
- 3.3. Undo/redo
 - 3.3.1. Undo and redo would improve the users experience dramatically
- 3.4. Debug console
 - 3.4.1. A debug console would make life easier as the developer
- 3.5. Saving
 - 3.5.1. The user should be able to save structures locally and to a remote on a database
- 3.6. Templates
 - 3.6.1. There should be a set of pre installed templates to help the user understand what can be done

Design

Description

The Project aims to create an intuitive, interactive and versatile sandbox for students to explore and share how physics is found in the real world. It will do this not by showing how it exists in the real world but instead by letting them discover that themselves. The Application is programmed in the C Sharp programming language and uses MonoGame at its core to handle all the graphics. With DearImGuiNet on top as the graphical user interface. The Project Will use object orientation across the project to increase readability and reduce code repetition. The User will interact with the project through a combination of windowed Interfaces, Direct interaction with the main game window and Keyboard Inputs.

The Application has a large range of Tools to interact with the simulation(or Mesh) that can be selected and modified through a menu. Each tool has a cursor that assists the user such as a circle that displays the area of effect of the given tool. Each tool also has any variation

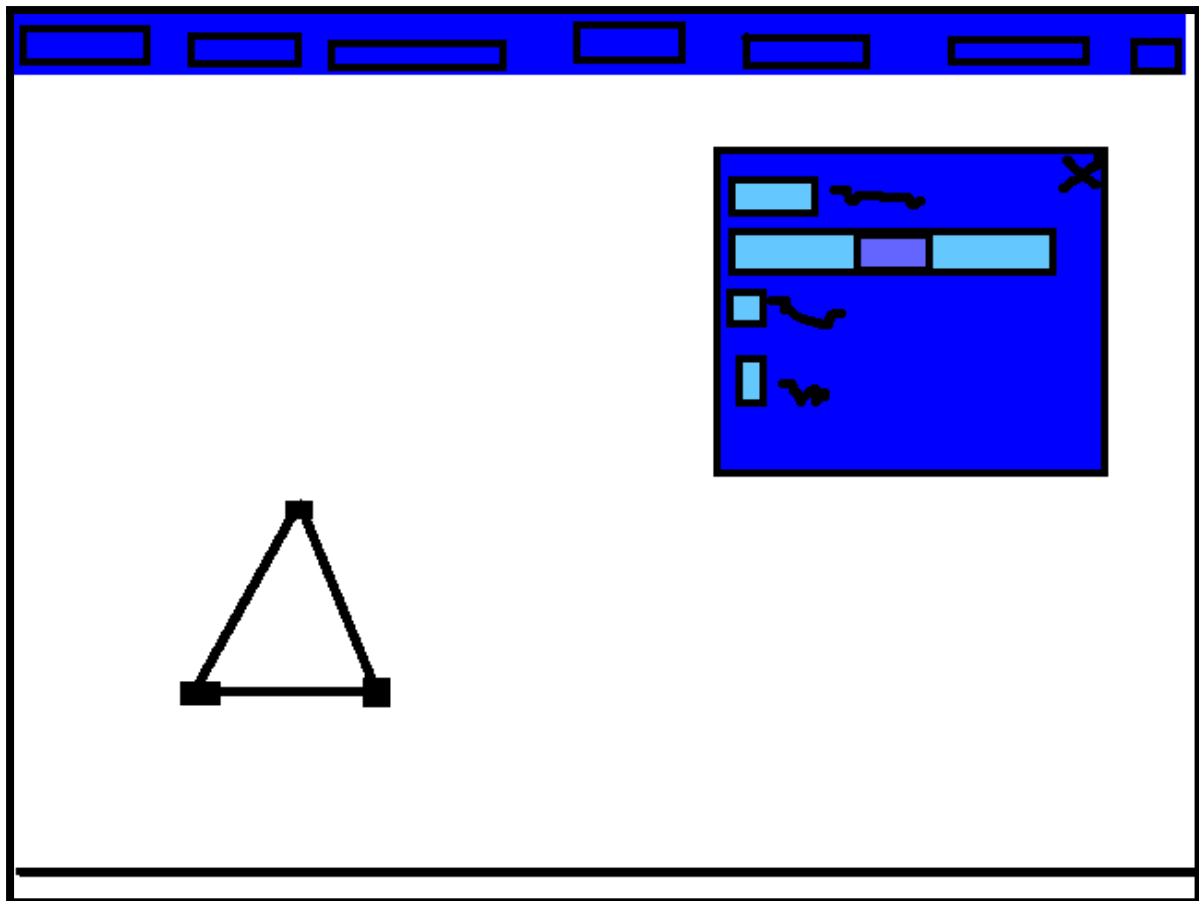
of properties stored in a Dictionary of the name of the property and the value stored. The users can emulate any environment by modifying the global variables through configuration window/menu.

For debugging of problems for the user or developer there is a debug console that contains a feed of logs and an input box that allows a user to type in commands and parameters this allows developers to quickly create functions without having to integrate with the GUI. This console logs everything that could possibly be relevant including general info, warnings and errors. The logs are collected automatically and numbered for repeated logs to reduce log clutter. The logs are labeled as Info, Warning and Error as to colour and filter in the console. The Log automatically saves to a file.

For work across multiple days or devices local and database saving is available. This is done by converting the Mesh to a simplified class and converting to a json File for saving locally and a json string for the database. The JSON file was chosen to make the save files readable and more resistant to updates as a binary file could completely break after changing the Mesh Class or changing platform if I ever wanted to port the program to the Web. The Database stores the users and information what type of user they are e.g. Teacher or Student, the Mesh and its relevant information such as owner id creation date and optionally Assignment ID. The Database also stores assignments set by teachers description due date and creation date. These assignments are used as more of a predefined sharing utility between students and teachers rather than homework.

User Interface Design

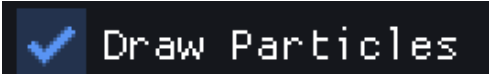
The User Interface will mainly be taken up by the simulation bounds with a menu bar where the user can either interface with drop down menus or openable windows.

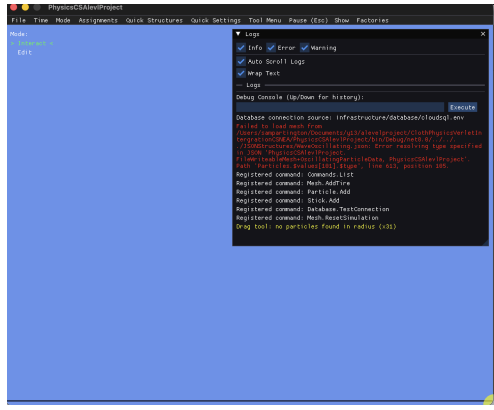


This is a rough design showing how the generic interface will look with the only constant interface that isn't the simulation taking up roughly 15% of the screen allowing for large simulations without restraints.

Design Rules

Instead of Designing every window i want to define when to

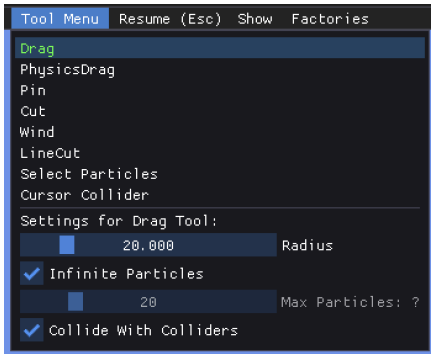
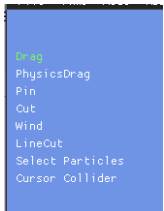
Name	Image	Description
Slider		Use the slider for when the user would want to change the given value often without set values as too tweak
Input Box with increments		For Values where exact values would want to be chosen but with slight tweaks.
Check Box		Used for Booleans should be obvious what is being turned on and off

Size of Window		The size of a window should stay to an absolute minimum especially for information that would benefit from being viewed or changed based on the reactions of the Simulation. I should aim for around a quarter of the screen be taken up as this leaves enough space to see the simulation
----------------	---	---

Specific Design Choices

Tools

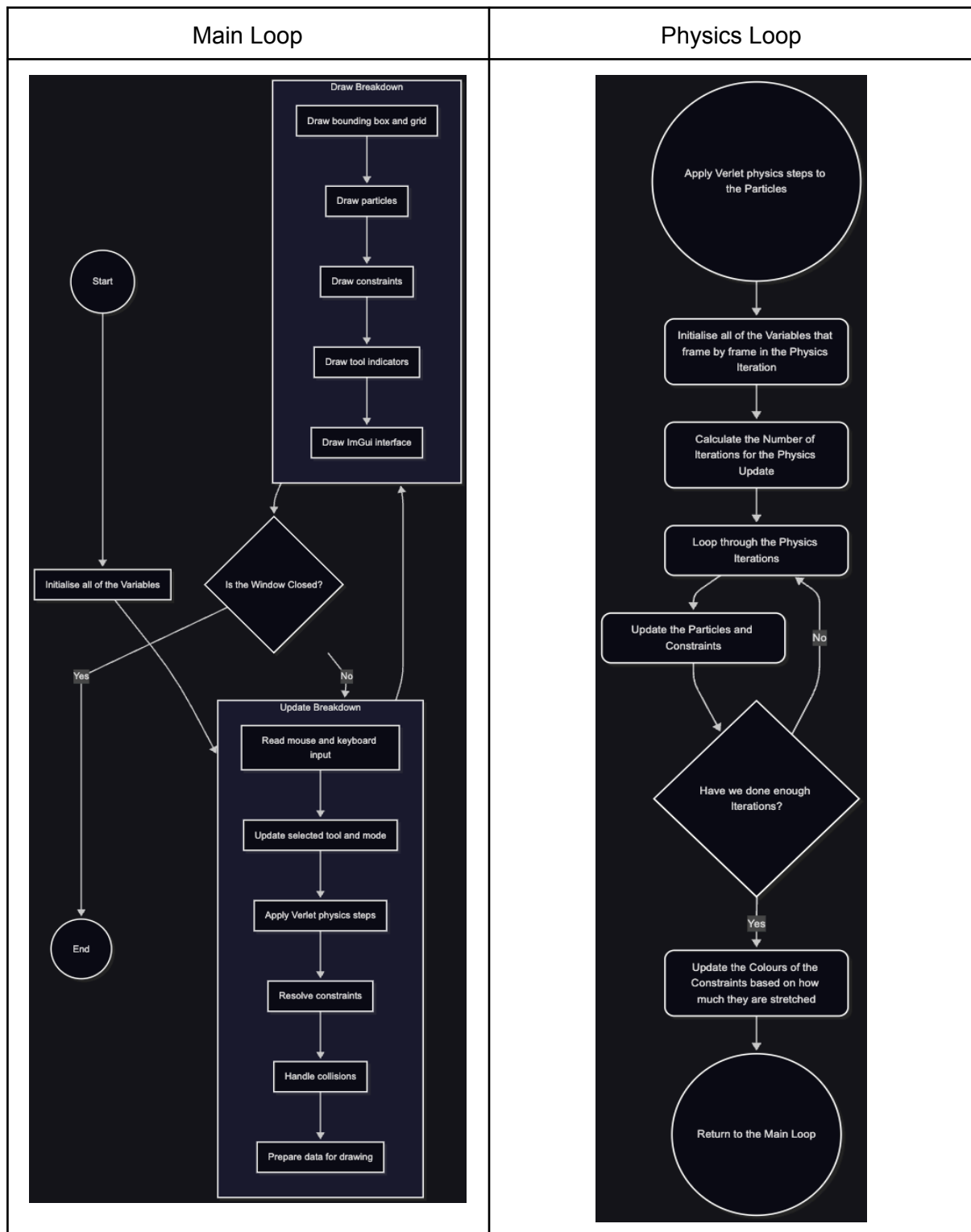
A central part of my project is the Tool System so the method for switching tools should be intuitive.

For any user to instinctively be able to select a tool a gui solution is a requirement. So at the top of the screen I will add a drop down menu that will be populated by the tools and its settings automatically through the tool dictionary.	
Though putting all of the tools in one space will become cluttered so I will add multiple modes that can be changed through the gui also.	
Another pillar of user interaction is keyboard shortcuts that allows seasoned users quickly work without the time spent interacting with a gui; a separate keyboard shortcut should be used for changing modes. The changing of mode or tool should be made apparent to the user through an overlay.	 

IPSO Chart

Input	Process	Storage	Output
Mouse and keyboard input, ImGui tool buttons, and selected tool settings such as drag, pin, cut, and physics options.	Validate the input, update the selected tool, apply Verlet physics, enforce constraints, and handle collisions.	Active mesh data, particle positions, stick constraints, and undo/redo history.	Updated mesh simulation on screen, tool feedback, and log messages.
Save/load button pressed on the structures screen, plus a chosen mesh name or database entry.	Convert the mesh to JSON, save it locally or to the database, and reload the selected structure when requested.	Local JSON mesh files, saved database structures, and the current active mesh.	Confirmation that the mesh was saved or loaded, and the restored mesh structure.
ImGui Buttons In a ImGui Window for Configuration of everything from simulation settings to Window Resolution	Validate the input and apply settings	Stored in the Mesh related data in the Mesh class for saving and more Client related data as general variables not saved	Updated mesh simulation and client changes. Changes sent to the logger
Open "Information window" pressed	Print all information with ImGui to do some of the formatting	Stored in the code a string	Prints the information on to a ImGui
Open "Assignment Teacher Window" pressed	Show the text boxes for creating an assignment. Get all current assignments and display them in tabs with there corresponding MESHes	Stored on the database	Updated imgui window
Open Logger	Check for what logger filters are pressed and add all relevant logs to the logs to be printed to a list.	Stored locally on disk and on memory	Print all logs in ImGui window

Flow Chart



Tools

To interact with the simulation the user will need an assortment of tools. These tools are split up into two categories to make navigation easier. Each tool also has its own custom cursor that will change depending on the situation to help the user

Interact Mode

Tool Name	Description
Drag	This tool drags all particles in a configurable radius. It works by teleporting all particles to a radius. Not to the cursor but relative to where it was initially selected. It is teleported every physics iteration rather than every frame.
Physics Drag	This drag works by applying classical verlet constraints to everything in a radius. The constraint's power is determined by three configurable values: Strength, Damping and MaxForce.
Pin	Pin tool selects all particles in a configurable radius; it works by setting the accumulated force from Sticks to zero and skipping all physics iterations.
Cut	Cut deletes all Sticks in a configurable radius
Wind	Wind applies a basic force to all particles like a temporary gravity. It is applied by clicking and dragging. With an arrow being drawn
Line Cut	Line cut cuts all constraints in a line. The line is drawn and applied by clicking a dragging
Select Particles	Select particles allows the user to select particles by clicking and dragging the same way you can on your desktop. When releasing the mouse and if there are particles in the square a window will open a window with a bunch of tick boxes. When toggling a tick box below the tickboxes you can inspect the exact information and data about the particle. You can also delete all selected particles with a button.
Cursor Collider	The cursor collider tool creates a collider attached to the cursor. This collider can be a rectangle or a circle with all relevant data

	being configurable
--	--------------------

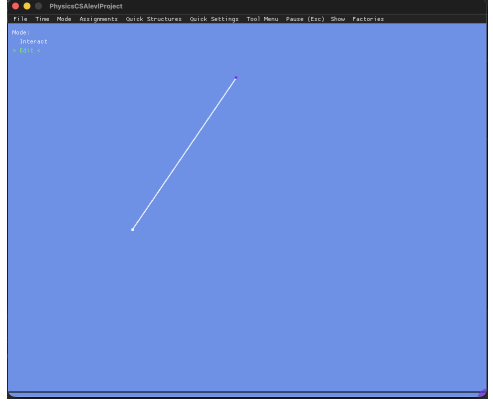
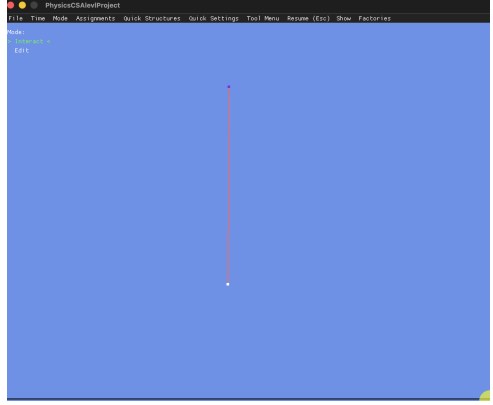
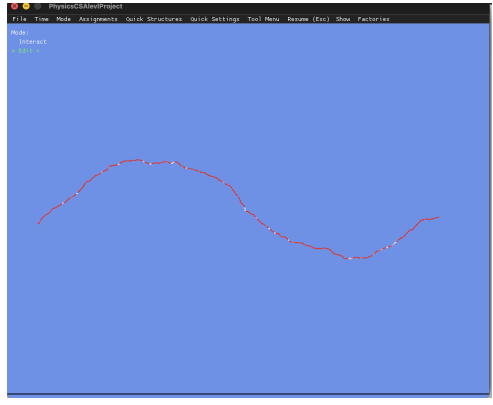
Edit Mode

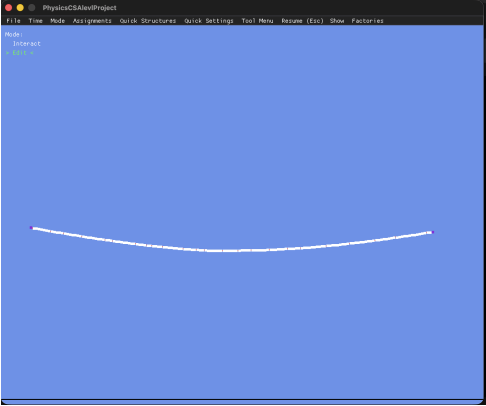
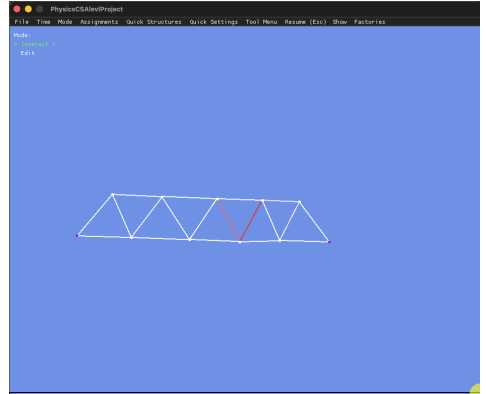
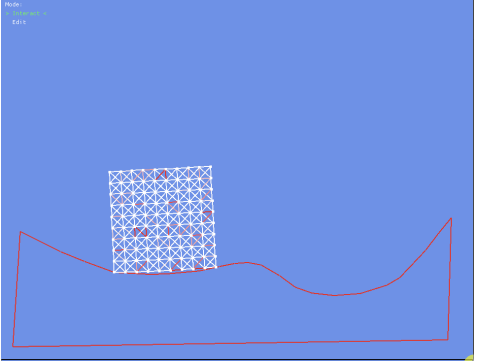
Tool Name	Description
Add Particle	Creates a particle on the mouse position when left mouse is pressed
Add Stick Between Particles	User can select two particles and on the second particle it connects the particles with a stick
Remove Particle	Delete particle in small radius on left mouse button
LineCut	Line cut cuts all constraints in a line. The line is drawn and applied by clicking a dragging
Pin	Pin tool selects all particles in a configurable radius; it works by setting the accumulated force from Sticks to zero and skipping all physics iterations.
Create Grid Mesh	Grid mesh allows you to create a grid. The grid density is determined by the "Stick Length" value which is configurable. It fills the box you draw with a grid on the press of "C"
Line Tool	Line tool creates a line of particles on sticks on the drag line when you release the mouse. It is configured by "Constraints in Line" which has a set number of constraints and automatically calculates the length of each stick and generates them. It is also configurable with Natural Length ratio. This allows you to change how stretched the sticks are when placed this is used to create taught lines of particles and sticks easily
Add Polygon	Add polygon allows the user to draw a polygon of sticks and particles by clicking. Each click places a new particle and connects the new particle with the previous particle with a stick. Enter is then pressed and this connects the newest particle with the first particle. The user can also press "C" to create a collider and delete the sticks and particles created. This allows the user to create a collider of any shape.
Oscillating particle	An oscillating particle creates a new particle that oscillates to a frequency. The amplitude and frequency is configurable as well as the angle the particle oscillates at
Place Collider	Allows the user to place two types of collider: the circle and the rectangle. The circle's radius is configurable. The Rectangles height and width is configurable as well as its angle
Move collider	Allows the user to click and move any collider including polygon colliders

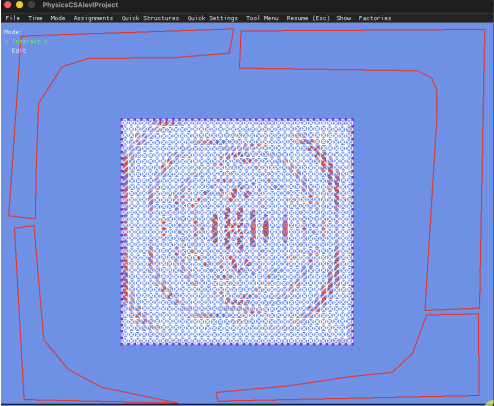
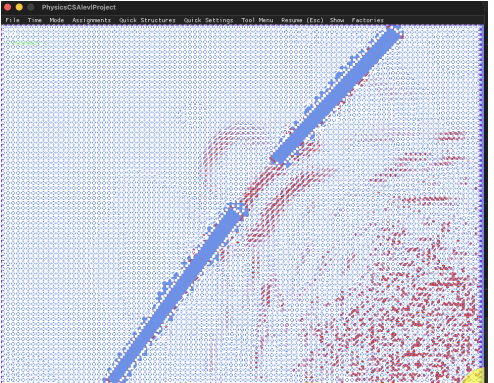
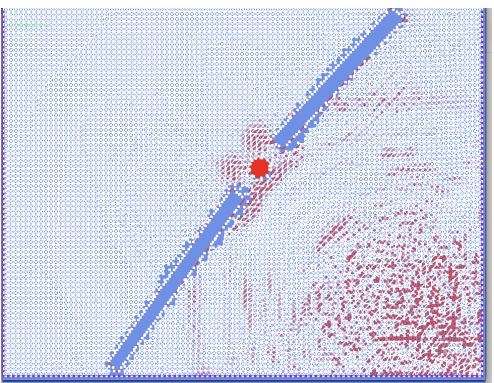
Delete Collider	Allows the user to delete a collider
-----------------	--------------------------------------

Templates/ Pre-installed Meshes

To help the user a selection of Meshes will be preinstalled to show certain simulations

Name	Description	Implementation
Pendulum	A basic pendulum in a zero friction environment with a very high spring constant	
Simple Harmonic Motion	A particle connected to a stationary pinned particle oscillating up and down with a low spring constant and zero friction	
Wave due to Oscillating particle	A line of taught particle and sticks connected to an oscillating particle simulates a transverse wave as well as	

Standing Wave	A line of taught particles and sticks connected to a oscillating particle with the spring constant tuned so that there is only a half wavelength	
Bridge	A bridge made from triangles emulating a truss bridge	
Soft Body	A grid of particles connected horizontally, vertically and diagonally so that the body holds its shape. This body resembles jelly and even skyscrapers displaying how everything, even objects that are seemingly rigid, have sway.	
Longitudinal wave	A line of particles connected to an oscillating particle that is oscillating parallel to the direction of energy transfer. As the Particle Oscillates back and forth more waves are made which interferes with the waves bouncing back causing constructive and destructive interference	

Grid Longitudinal wave	A grid placed in the same way as the soft body with all of the edge particles pinned. The user can then apply a force by dragging the middle to initialise the wave. This shows a ripple effect emulating water and rain drops.	
Diffraction pattern	By filling the screen with a full grid cutting off sections as shown in the image on the left. On most devices there are too many particles so time stepping is the only option. You can see how the Wave gets constricted at the slit and curves out .	
Double slit diffraction pattern	The normal diffraction pattern with a collider in the middle to create a double slit you can see the waves interfering with themselves and cancelling each other out.	

Vector Graphics Library

To make drawing stuff easy I wrote a library that makes drawing primitives easy in monogame as there wasn't an easy way natively. The main three shapes that I used in my project were the Circle, The rectangle and the Line. Each shape inherits from a base Shape class in case a polymorphic use-case was required.

The generation of the shapes did not use shaders but instead initial Texture generation. The circle for instance uses Bresenham's circle generation at launch to in the white colour so it can be recoloured and resized to anything i wanted. The line and rectangle textures are much more simple. They use a single white pixel and stretched and rotated in the spriteBatch draw to efficiently create shapes. The line shape was widely used as it made drawing lines extremely easy by either two points and doing the maths to join them or by having a single point a length and an angle to create a line.

Mesh

The entire simulation is stored in a class called Mesh; it contains everything that would be relevant to a mesh. Including the Colliders, sticks and particles. The mesh class does not contain any functions related to drawing or physics. It only contains functions related to creating structures in the mesh to be used by tools or console commands.

Particles

The central part of the simulation is the particles. They are each stored as a class called "Particle". The Particle class stores the current position, previous position and accumulated force for the physics calculations. To easily identify and reference the particles in the update facing code each particle is given a unique Identifier. The particle class also stores the IsPinned boolean which is used to check if the particle's physics calculations need to be skipped. To make finding sticks connected to particles easier a Dictionary containing the Identifier for each particle as the key and a hashset containing the IDs of all connected sticks as the value. This reduces finding the connected sticks from a time complexity of n to a time complexity of 1.

Sticks

Sticks are relatively simple; all they store is the two IDs of the particles they are connected to and the "natural length" which is just the length of stick that the simulation will try and resolve towards.

Drawing

To keep drawing simple I extended the Particle and Stick class with Drawable variants of them both. They both use my drawing library. The Particle uses the Primitivebatch.Rectangle and the Stick uses primitivebatch.line.

The sticks change colour based on how stretched they are. The variance in colour is determined by a normally distributed linear interpolated colour between Red and white. This is used to show stress and the transfer of energy.

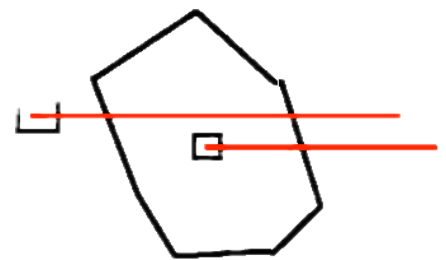
Physics Iteration

To increase accuracy and stability the simulation does multiple iterations of physics per second. To prevent the simulation from speeding up Δt (the time between frames) is divided by the number of iterations and is multiplied in the verlet integration calculation. The number of substeps is if configurable via the quick settings menu though higher values are buggy and laggy and too low values are also buggy but not laggy so changing the substeps to maximize the accuracy of your simulation is recommended

Colliders

The collider is a base abstract class that contains the collider position as the only base variable. For subroutines it has the ContainsPoint This function returns a bool which determines whether a point is contained within the collider. It has two inputs the vector2 point and the out vector2 closestPoint. It also contains the drawing function for drawing the collider and the DeepCopy function for deferencing. The Mesh class stores a list of the Collider class as to polymorphism the different colliders in one contained iterable list.

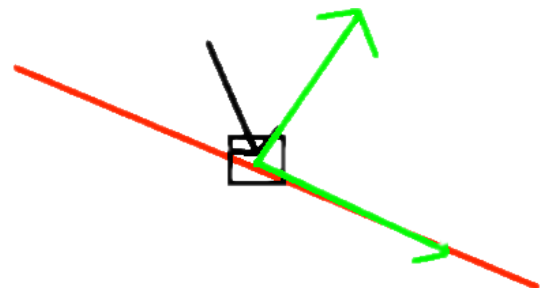
There are three types of colliders that all inherit the collider class. The circle only contains a radius as an extra variable. The Rectangle stores the height, width and angle. And finally the most complicated Polygon collider. This is a collider that allows the custom creation of any non self intersecting shape convex or concave. It uses Raycasting to perform the collision detection. This works by sending out a ray on the positive X direction and counting how many faces the ray collides with. If the number of intersections is odd the point must be inside of the polygon and if it is even it must be outside



To improve performance all of the colliders use something called broadphase. It generates an exterior rectangle containing the more complicated collider and checks if the particle is inside the axis aligned rectangle before doing the more complex work.

Collision Resolution

To keep collisions realistic the velocity is rebounded. The direction is calculated via the difference of the position before the collision and after. The resolved velocity is split into two separate velocities: the normal velocity and tangential or the bounce and the friction velocity. This allows for realistic materials.



Undo / Redo

To allow the user to un fearfully edit and manipulate the mesh in the simulation a Undo and Redo system was implemented. It uses two Stacks one for Undo and the other for redo Each with the "Mesh" class as the type. This allows me to easily decide what things should be included when undoing and redoing. When pushing the current "Mesh" the DeepCopy must be pushed otherwise only the reference is pushed which causes problems when

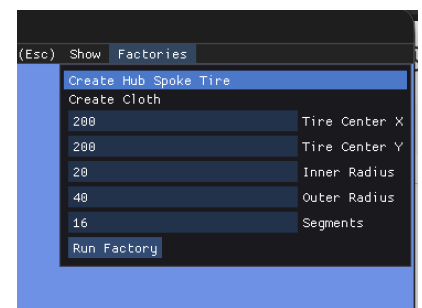
loading different meshes through quick structures. When undoing the program pushes the active mesh to the Redo stack and consequently pops the undo stack. The action is logged and the simulation is paused. The Mesh is pushed to the Undo stack at every reasonable action including when the simulation is paused.

MyStack

The stack used in the Undo and Redo system is hand written not the built in c sharp one. It does not use a linked list to store the data instead it uses an array to save memory. This does not mean the size of a stack has a limit. On creation of the stack the array is set to have a size of 100. When pushing to the stack the program compares the `_top` value with the length -1. If the values are equal the program copies the current array to a new array with double the length. This allows for fast memory efficient stacks without compromising on size limits.

Factories

To help the user create more complicated structures like the Tire i wrote a Factory system. Factories use a very similar system to commands but are implemented in a GUI. There is a base Factory class. It contains the Action type with `Object[]` as its input. This allows me to easily input any method into a factory. Currently I only extend the Factory class to two factories. Tire and cloth. They both contain a Dictionary with the names of arguments as the key and default values as the value. The Factories are stored in a Factory List. This allows the program to automatically generate the GUI for any future factories.



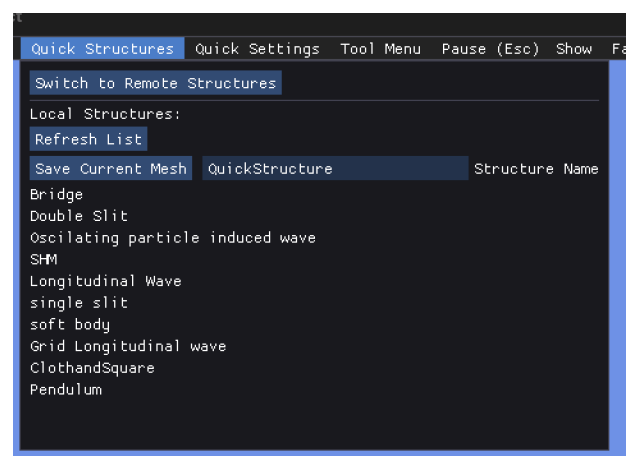
```
public class Factory
{
    2 references
    public Action<object[]> Method { get; }
    2 references
    public string Name { get; }
    5 references
    public Dictionary<string, string> Parameters { get; }

    2 references
    public Factory(string name, Action<object[]> method, Dictionary<string, string> parameters)
    {
        Name = name;
        Method = method;
        Parameters = parameters;
    }
}
```

File Saving

To reduce the size of files and increase the stability of the Structures the Mesh class is abstracted to a simpler filewritable mesh class with filewritable particles and Sticks to be paired with them stored inside. Then to convert the class to a file I use the

`Newtonsoft.Json.Serialization;` library which allows for easy conversion of classes to a JSON file and vice versa the JSON is outputted as a string to be saved to either the database or to a file. The files are loaded in at boot and every time a new file is added. The program iterates through the JSONstructures directory deserializes each json file and puts them into memory.



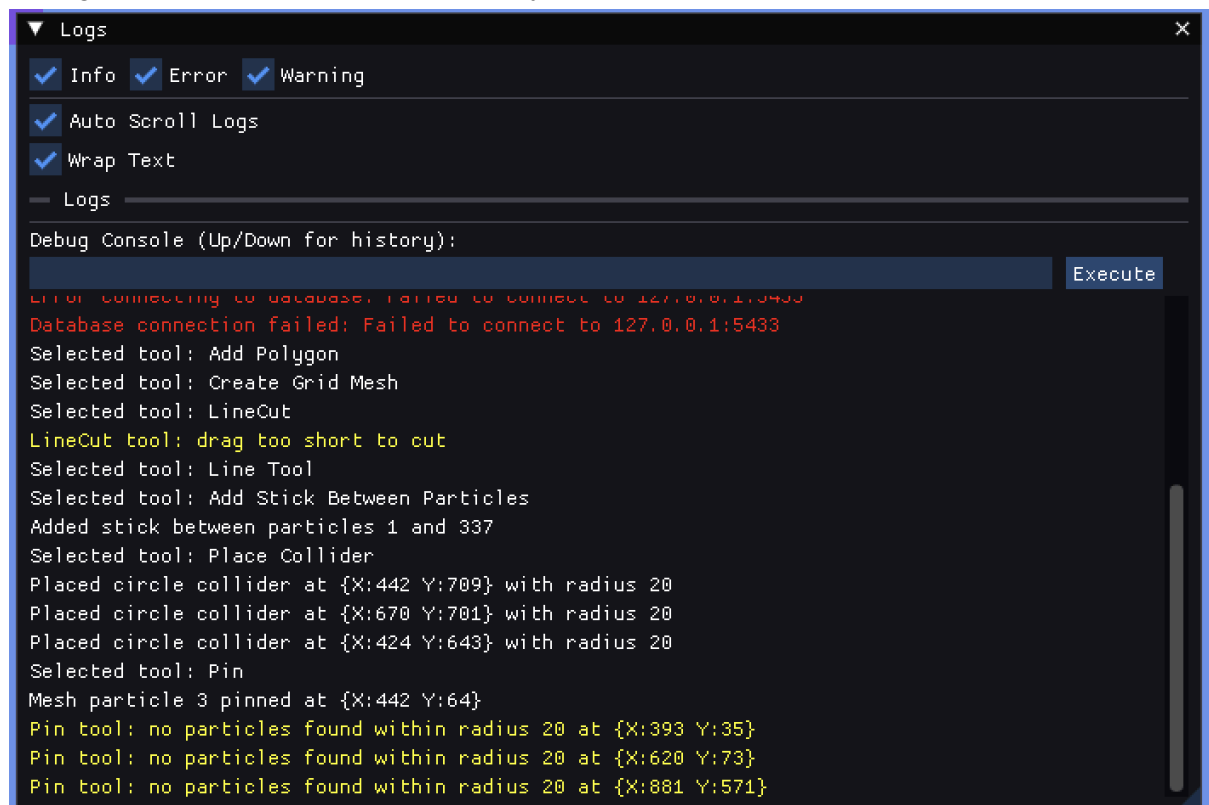
Logger and Debug Console

As to assist my development I wrote a class called ImGuiLogger that creates an ImGui window with a log and a box to input commands

Log Design

In the program the developer at any point can push a log with a tag. The tags describe the colouring and allow the user to filter out the different tags. The tags are “Info” “Warning” and “Error” they are stored as an enum for readability. These logs are also saved when the application closes to a log file. To share in case of problems. The Logger automatically collapses repeated logs and marks the number of repetitions at the end of the log entry so the user isn’t overwhelmed with a page full of repeated logs.

The logger has allowed for safer exception catching as the errors can be safely pushed to the log queue marked in red so it is easily found and filtered out.



Debug Console

During development I found the process of constantly creating a gui for every feature a slow process and decreased productivity so I wrote a Debug console that uses something called Reflection. Reflection allows a developer to mark a function and it will be able to be called in the console. This is much faster than the alternative of creating a switch case tree and much more readable. The function does need to be set up in a certain way though. As all reflected functions are listed in plain text it also allows for tab autocompletion in the future. For its inputs you need to keep everything the same so an array of Strings is needed. To get these parameters and commands the Logger separates the command using string manipulation into a class called Command that stores the main command and arguments. It also needs

to resolve the environment variables. All commands are stored in a list so the user can use the arrow keys to retrieve previously executed commands.

Environment Variables

I decided to implement an environment variable system. This allows the user to reference something like the mouse x and y in the command like "\$mouseX" or "\$windowH" they are found and replaced efficiently with regex before converting the command string to the Command class

Database

The Database needs to store 4 main things: the users, the user roles, the meshes and assignments. This is stored in postgresSQL and it will be handled by the Game1Database Class. This Class acts as an intermediary for the rest program.

Entity Descriptions

Roles

Field	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	An Auto incrementsing unique ID
name	role_type	UNIQUE, NOT NULL	An ENUM with two values "Student" and "Teacher"

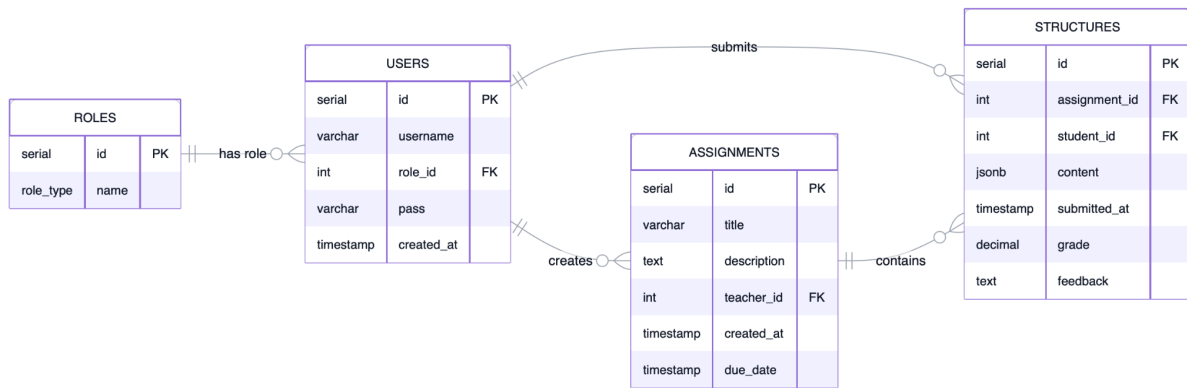
Users

Field	Type	Constraints	Description
ID	Serial	PRIMARY KEY	An auto incrementing unique ID
Username	VarChar(50)	NOT NULL, UNIQUE	The display name of the user
Pass	VarChar(255)	NOT NULL	The password for the user. Currently stored as plaintext for testing purposes in production it would be hashed
Role_ID	int	FOREIGN KEY, ROLES(ID)	The foreign key linking the user and role
Created_at	TimeStamp	DEFAULT_CURRE NT_TIMESTAMP	Auto setting timestamp

Structures

Field	Type	Constraints	Description
id	SERIAL	PRIMARY_KEY	An auto incrementing unique ID
assignment_id	INT	FK -> Assignments(id), ON DELETE CASCADE	The foreign key linking the Structure and role
student_id	INT	NOT_NULL,FK -> ON DELETE CASCADE	The foreign key linking the user and role
content	JSONB	NULLABLE	Stores the complete json file. The JSON type allows for future functionality where the user can search for specific things in the JSON
submitted_at	TIMESTAMP	NULLABLE	The time for when the structure is submitted to an assignment
grade	DECIMAL(5,2)	NULLABLE	Reserved for a future feature so that teachers can grade structures
feedback	TEXT	NULLABLE	Reserved for a future feature so that teachers can give feedback on the Structure
UNIQUE	(assignment_id,student_id)		Ensures each student can only have one submission per assignment

Entity Relationship Diagram



Testing

 FinalVideo.mp4

Test Num	Type	Description	Test Data/ Action	Expected Result	Actual Result	Pass/ Fail	Evidence
1	Normal	Verlet Integration Works properly under gravity	Place particles without any constraints with default settings	The particles fall downward at a normal speed and acceleration and come to rest	worked as expected	Pass	2:40
2	Normal	Particles are constrained by boundary walls	Fling particles at the all 4 walls	The particles collide and come to rest the the direction of the wall	worked as expected	Pass	2:44
3	Normal	Sticks Attempt to maintain Natural Length	Place a two particles connected with a stick in zero gravity and then nudge using drag	the stick will apply forces on both particles evenly to keep the natural length	worked as expected	Pass	2:57
4	Normal	Particles collide with all types of colliders	place each type of collider and drop a particle on each	the particle collides with the collider and resolves collision	worked as expected	Pass	3:26
5	Normal	Drag Tool moves particles with cursor	Left Click Drag on Cloth Mesh	Selected Particles move with the cursor relative to the cursor	worked as expected	Pass	3:51
6	Normal	Pin Tool freezes particles	place a particle and use the pin tool in normal gravity	the particle stays in place in spite of gravity	worked as expected	Pass	3:59
7	Normal	Cut Tool deletes all sticks in radius	apply the cut tool in the middle of the cloth mesh	all particles in radius become disconnected	worked as expected	Pass	4:09

			the the rest of the cloth and fall			
8	Normal	Line Cut deletes all sticks in a line	Let mouse drag and release through across the cloth mesh	the bottom half of the cloth falls as its no longer connected the the pinned top half	worked as expected	Pass 4:35
9	Normal	Wind Tool applies force in direction of wind	Left click drag with various polygons in the window	all the polygons move in the direction of the click and drag	worked as expected	Pass 4:46
10	Normal	Physics Drag	Left Click Drag on Cloth Mesh	the selected particles follow the cursor dragging behind	worked as expected	Pass 5:17
11	Normal	Cursor Collider interacts with particles	move the cursor through the mesh	the particles collide with the cursor and resolve that collision	worked as expected	Pass 5:28
12	Normal	Add Particle	Left Mouse in the window	a standard particle is added	worked as expected	Pass 5:40
13	Normal	Add Stick Between Particles	Left Mouse on two separate particles	a stick is added between the two particles with the natural length being equal to the initial distance between the two particles	worked as expected	Pass 5:46
14	Normal	Remove Particle	Click On a particle in the cloth	particle is deleted with all stick connects removed	worked as expected	Pass 5:54

15	Normal	Create Grid Mesh	Click and drag in a diagonal fashion then press "C"	A grid of particles all connected with sticks is created	worked as expected	Pass	6:03
16	Normal	Line Tool	Click Drag Release for a reasonable distance	a line of particles and sticks are created all connected	worked as expected	Pass	6:12
17	Normal	Add Polygon(Particle Stick)	click multiple times then press "Enter"	particles are created and connected to the previous particle except for the initial particle. when enter is pressed the final particle is connected to the initial particle	worked as expected	Pass	6:25
18	Normal	Add Polygon(Collider)	Click multiple times then press "C"	all particles created while making the polygon in preparation for the Particle Stick Polygon are deleted and in its place a collider is created	worked as expected	Pass	6:33
19	Normal	Place Collider	Select a Collider Configuration then left mouse in the window	When Left Mouse the selected collider is spawned on the cursor position in the correct configuration	worked as expected	Pass	6:50
20	Normal	Move Collider	Click and drag on a collider	the collider moves with the mouse	worked as expected	Pass	7:01

				relative to where it was clicked			
21	Normal	Delete Collider	click on a collider	the collider is removed	worked as expected	Pass	7:04
22	Normal	Change Tool With Menu	go the menu bar and click/hover and on the tool Menu then click on a tool	The tool changes to the selected tool and the cursor changes to the correct tool cursor	worked as expected	Pass	7:08
23	Normal	Change Tool With Keybind	Press "Ctrl" "T"	the selected tool cycles	worked as expected	Pass	7:12
24	Normal	Change Mode With Menu	Press the Mode Menu Button and select a mode	The mode changes and the tool menu updates to the new mode	worked as expected	Pass	7:24
25	Normal	Change Mode With Keybind	Press "Shift" "Tab"	The mode changes and the tool menu updates to the new mode	worked as expected	Pass	7:28
26	Normal	Save Mesh to local file	open the quick structures menu and name and save the current mesh	the current mesh is saved to a JSON file named the name given in the JSONStructur es directory	worked as expected	Pass	7:32
27	Normal	Load Mesh from Local file	click a previously saved Mesh and press load	the previously saved mesh is loaded into the simulation the exact same way it was saved	worked as expected	Pass	8:04
28	Normal	Switch mode	Switch to edit mode	the simulation pauses	worked as expected	Pass	8:19

29	Normal	Sign in to a user account	press file then sign in fill in the User with "Bob_builder" and password with "password"	the window displays "signed in successfully"	worked as expected	Pass	8:29
30	Normal	Sign out	press file then sign in and click sign out	the window displays sign in information	worked as expected	Pass	8:41
31	Normal	Save Mesh to database	press quick structures then switch to database name and press save	the mesh saves to the database	worked as expected	Pass	9:05
32	Normal	Load Mesh from Database	press quick structures then switch to database name and select a structure	the previously saved mesh is loaded into the simulation the exact same way it was saved	worked as expected	Pass	9:20
34	Normal	Open Show ReadMe Window	Press show then Reame	Readme window is opened with all information printed	worked as expected	Pass	9:21 9:35
33	Normal	Quick Setting in quick settings applies to the simulation	change a value like BaseForce(gravity) with the slider	the effect applies in the simulation	worked as expected	Pass	9:27
35	Normal	Time Step	press step forward while the simulation is paused	the simulation steps forward the given amount of frames	worked as expected	Pass	9:45
36	Normal	Reset Simulation	press reset simulation in the time menu	all of the particles and sticks clear leaving the colliders	worked as expected	Pass	10:03
37	Normal	the logger logs stuff in the window	press show then logger	the logger window is opened and prints all logs	worked as expected	Pass	10:14

				and colours them			
38	Normal	the logger prints the logs	press drag in empty space to trigger a log being sent to the logger	the logger prints a yellow warning log describing that no particles are printed	worked as expected	Pass	10:14
39	Normal	the logger can execute commands	open the logger and type Particle.Add(200, 200,0.1) and press enter	a particle is created at 200,200	worked as expected	Pass	10:26
40	Normal	the logger can filter logs	deselect every tick box but one	only the logs of the selected type are printed	worked as expected	Pass	10:32
41	Boundary	test the maximum subscripts	in quick settings drag the slider to the max	the simulation performs more substeps and accounts for the increase in iterations with deltatime	due to a bug most likely to do with dividing small floats the polygons appear possessed is not useful for any sort of simulation	Fail	10:40
42	Boundary	test the minimum substeps	drag the substep slider to the minimum	the simulation is less stable and less accurate	at one substep the simulation immediately breaks and all of the particles exploder apart probably due to a bug but i am insure what it could be	Fail	10:49
43	Normal	Set an assignment as a teacher	sign in as ms_engineering and create an assignment using the teacher assignment window	the assignment is visible to the to other students	worked as expected	Pass	10:56

44	Normal	Save a structure to an assignment as a student	as a student click assignments then a teacher then their assignment and save the current structure to one with a name	the structure is visible to the teacher that owns the assignment	worked as expected	Pass	11:35
45	Normal	open a student's assignment as a teacher	go to assignments then browse as teacher then click on the assignment then load a structure made by a student	the structure loads in the the simulation window	worked as expected	Pass	11:55
46	Normal	Undo and redo	perform an action then undo it and then redo it	When undoing it goes back to the previous action that would warrant a undo history stack push it should keep all data including velocities and such . when undoing/redo ing it should also pause the simulation	worked as expected	Pass	12:25

Fixes

Test	Issue	Solution	Resolved
41-42	Physics breaking at the extreme substeps	To fix this bug it would require a massive change in the way physics iteration works so I will	

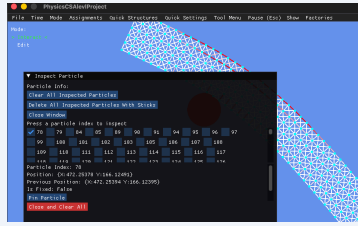
		use the most basic fix of capping the substeps to safe values. The bug was caused by dividing extremely small numbers causing imprecision and instability. Before Fix: <pre>ImGui.SliderInt("Physics Substeps", ref _subSteps, 1, 600);</pre> The Fix: <pre>ImGui.SliderInt("Physics Substeps", ref _subSteps, 10, 120);</pre>	
--	--	---	--

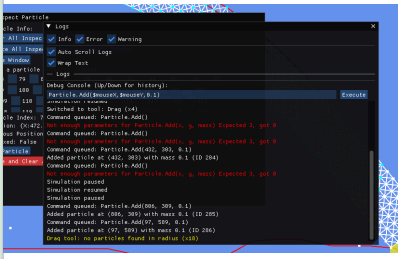
Evaluation

Objectives

Objective number	Objective	Implemented?	Evidence	Notes
	1. Mesh			
	1.1 Mesh Representation			
1.1.1	Custom Mesh class stores all simulation data (particles, sticks, colliders, physics settings)	Yes	Objects.Mesh.cs	The single class is used for storing in my custom stack for undo and redo functionality
1.1.2.1	Stick class stores natural length and IDs of connected particles	Yes	Objects.Stick.cs	Natural length set at creation from particle positions
1.1.2.2	Particle class stores current position, previous position, accumulated force and mass	Yes	Objects.Particle.cs	Previous position used by Verlet integration instead of velocity
	1.2 Core Mesh Functions			
1.2.1	Add Particle	Yes	Test 12	Adds a particle on click at mouse position
1.2.2	Add Stick Between Particles	Yes	Test 13	Adds sticks when clicking on two particles
1.2.3	Remove Particle	Yes	Test 14	Also removes all particles in a configurable radius automatically disconnects all sticks
1.2.4	Remove Stick Between Particles	Yes	Test 7	Deletes all stick connects in a configurable radius
1.2.5	Deep Copy for multiple mesh instances (undo/redo and save)	Yes	Objects.Mesh.cs	This is heavily used as to stop accidentally referenced classes
	1.3 Mesh Builder Helper Functions			
1.3.1	Grid generator — creates grid of particles and sticks, pins top row	Yes	Test 15	Density configurable via Stick Length setting
1.3.2	Polygon builder — create closed polygon via mouse clicks	Yes	Test 17	Enter closes shape after clicking at least 3 times
1.3.3	Line generator — line of particles with configurable density	Yes	Test 16	Constraints In Line settings controls the number of particles and how taught the line is
	1.4 Mesh Drawing			

1.4.1	Drawing functions on Particle and Stick classes	Yes	Video-[0:00]	Extends base classes; uses PrimitiveBatch to draw efficiently
1.4.2	Colour particles and sticks based on user interaction and world state	Yes	Video-[0:00]	Sticks coloured by stretch amount. particles by pin/select state
2. Physics				
2.1	Core physics iteration for particles using Verlet integration	Yes	Video-[0:00]	$x(n+1) = 2x(n) - x(n-1) + a \cdot dt^2$; pinned particles skipped
2.2	Core physics iteration for sticks — constraint resolution applied to particles	Yes	Video-[0:00]	Hooke's law spring force accumulated per particle per sub-step force = $K \cdot \text{change in length}$
3. User Interaction				
3.1 GUI				
3.1.1.1	Information / ReadMe window explaining controls and current state	Yes	Test 34	Shows mode, tool, user and keyboard shortcuts and current mesh data
3.1.1.2	Menu for switching tool and mode	Yes	Test 22	Drop-down auto-generated from tool dictionary
3.1.1.3	Settings / configuration menu	Yes	Test 33	Resolution, gravity, sub-steps, spring constant etc.
3.1.1.4	Menu for saving and loading meshes locally and to database	Yes	Test 25 and 26	Local JSON and remote PostgreSQL both supported
3.2 Edit Mode Tools				
3.2.1	Build polygons through mouse inputs	Yes	Test 17	Click to place particles; Enter to close
3.2.2	Build lines of particles and sticks	Yes	Test 16	Drag to define line; release to generate
3.2.3	Delete and add particles	Yes	Test 12 and 14	Use the tool system
3.2.4	Add sticks between particles	Yes	Test 13	Click first then second particle
3.2.5	Place circle and rectangle colliders	Yes	Test 19	All relevant information configurable.
3.2.6	Move colliders	Yes	Test 20	Click and drag any placed collider realtime
3.2.7	Delete colliders	Yes	Test 21	Deletes on click
3.2.8	Oscillating particle to drive waves	Yes	Video[1:39]	Frequency, amplitude and angle configurable though not changeable after placable

3.2.9	Custom polygon colliders via mouse	Yes	Test 18	Uses ray casting for colliding
3.3 Interact Mode Tools				
3.3.1.1	Drag — teleport particles in radius to cursor	Yes	Test 5	Applied every physics sub-step not every frame with linear interpolation
3.3.1.2	Physics Drag — apply spring force to particles in radius	Yes	Test 10	Strength, damping and MaxForce configurable
3.3.1.3	Drag radius fully adjustable	Yes	Video [0:39]	Radius slider in tool settings
3.3.2.1	LineCut — cut all sticks intersected by a drawn line	Yes	Test 8	Click and drag to cut in the line
3.3.3.1	Select Particles — display position and accumulated force	Yes		Click-drag selection box, data shown in popup window
3.3.4.1	Pin tool — freeze particles, skip physics	Yes	Test 6	Sets IsPinned. accumulated force zeroed each tick and physics is skipped
3.3.5.1	Cut — delete sticks in a configurable radius	Yes	Test 7	
3.3.6.1	Wind — apply temporary directional force to all particles	Yes	Test 9	Click and drag sets direction and magnitude. arrow drawn on screen using primitives
3.3.7.1	Cursor Collider — collider locked to mouse cursor	Yes	Test 11	Circle or rectangle. dimensions configurable
3.4 Undo / Redo				
3.4.1	Undo last mesh change (Ctrl+Z)	Yes	Test 46	Deep copy of mesh pushed to history stack on every edit
3.4.2	Redo undone change (Ctrl+Shift+Z)	Yes	Test 46	
3.5 Debug Console				
3.5.1	Console accepts typed commands with parameters	Yes	Test 39	Commands registered via reflection discovered at startup.

3.5.2	Environment variable substitution (\$mouseX, \$mouseY etc.)	Yes		Replaces the environment variable text with the number with regex
3.5.3	Navigable command history (arrow keys)	Yes	Video[2:09]	Useful to prevent repeated typing
3.5.4	Filterable log output by level; auto-saved to file	Yes	Test 40	Info / Warning / Error repeated logs collapsed with count
4. Saving				
4.1.1	Save and load mesh to local JSON file	Yes	Test 24 and 26	JSONB chosen for human readability and update resilience but mostly because it's easier to implement
4.1.2	Save and load mesh to remote PostgreSQL database	Yes	Test 44	
4.1.3	User sign in with teacher or student role	Yes	Test 43	Role stored in Roles table; drives UI availability
4.1.4	Teachers can create assignments with title, description and due date	Yes	Test 43	
4.1.5	Students can submit a mesh linked to an assignment	Yes	Test 44	UNIQUE(assignment_id, student_id) prevents duplicate submissions
4.1.6	Teachers can view and load student submissions	Yes	Test 45	
5. Templates				
5.1	Pre-installed meshes demonstrating SHM, waves, soft body, bridge, diffraction	Yes	Video[1:30]	Pendulum, SHM, Transverse Wave, Standing Wave, Soft Body, Bridge, Longitudinal Wave, Grid Wave, Diffraction, Double Slit

Conclusion

I have sufficiently complied with my objectives put in place

User Feedback

To help find where I can improve I got some beta testing from a peer:

“Your soft body physics simulation is an impressive and well-thought-out project, particularly in how effectively it bridges computer science and physics concepts for A-level students. One of its strongest aspects is the simulation itself, which is both engaging and educational. It clearly demonstrates how soft bodies are constructed from particles and how those particles interact with rigid surfaces, making abstract ideas much easier to visualise and understand.

The inclusion of comprehensive supporting materials, such as keybinds and a ReadMe, shows strong consideration for usability. This makes the program accessible even to new users, which is especially important given its educational purpose. Additionally, the variety of preset structures is a significant strength. These presets not only showcase the capabilities of the simulation but also guide users toward meaningful exploration without requiring them to build everything from scratch.

A particularly notable feature is the way the program extends beyond basic soft body physics to simulate wave behaviour. The inclusion of longitudinal and transverse wave presets, as well as single and double slit experiments, demonstrates ambition and a clear link to A-level physics content. This helps users see how foundational principles can scale into more complex phenomena, reinforcing deeper understanding.

One area for improvement is stability and user experience. Currently, the program can crash if certain quick structures, such as the single and double slit experiments, are activated while the simulation is running. This could be addressed by restricting these features to only function when the program is paused or by implementing safeguards to prevent unintended use. Improving this aspect would make the program more robust and user-friendly.

Overall, this is a strong and effective educational tool that successfully combines technical implementation with clear learning outcomes. With minor refinements to stability and error handling, it has the potential to be an excellent resource for A-level physics students.”


Zackariah Wood (10 MGR)

To: Leon Partington

👤 🔍 📧 ...

Thu 30/04/2026 13:49

Your soft body physics simulation is an impressive and well-thought-out project, particularly in how effectively it bridges computer science and physics concepts for A-level students. One of its strongest aspects is the simulation itself, which is both engaging and educational. It clearly demonstrates how soft bodies are constructed from particles and how those particles interact with rigid surfaces, making abstract ideas much easier to visualise and understand.

The inclusion of comprehensive supporting materials, such as keybinds and a ReadMe, shows strong consideration for usability. This makes the program accessible even to new users, which is especially important given its educational purpose. Additionally, the variety of preset structures is a significant strength. These presets not only showcase the capabilities of the simulation but also guide users toward meaningful exploration without requiring them to build everything from scratch.

A particularly notable feature is the way the program extends beyond basic soft body physics to simulate wave behaviour. The inclusion of longitudinal and transverse wave presets, as well as single and double slit experiments, demonstrates ambition and a clear link to A-level physics content. This helps users see how foundational principles can scale into more complex phenomena, reinforcing deeper understanding.

One area for improvement is stability and user experience. Currently, the program can crash if certain quick structures, such as the single and double slit experiments, are activated while the simulation is running. This could be addressed by restricting these features to only function when the program is paused or by implementing safeguards to prevent unintended use. Improving this aspect would make the program more robust and user-friendly.

Overall, this is a strong and effective educational tool that successfully combines technical implementation with clear learning outcomes. With minor refinements to stability and error handling, it has the potential to be an excellent resource for A-level physics students.

The physics student's feedback was largely positive with the student identifying the simulation's usefulness as an educational tool and explicitly praising the variety of built in

structures that link back to Physics. Particularly the Wave Presets that demonstrate clear links to the physics A-level specification.

The Student noted the readme window and keybinds made the program accessible to new users which is incredibly important for any program. The built in wave structures are highlighted as a particular strength showing that the particle-spring model can be manipulated to model a wide variety of concepts. From simple truss bridges to diffraction patterns showing how on the micro scale everything acts like springs and particles.

However the tester identifies a stability issue when loading large presets such as the single slit and double slit experiments. The program froze while loading and converting the structures from JSON to the Mesh class. And when running them realtime accidentally it became extremely difficult to then pause and iterate manually so force closing the program is necessary. This obviously is a large problem as it breaks the users workflow and should not happen in the production version.

This happened due the rendering , Physics and file handling all being on one thread meaning if the physics or file handling has a large problem that will take longer than it takes to render one frame then the program will slow down.

To fix this I would put the low compute cost rendering on a separate thread to the rest of the work that would guarantee stable framerate even with complex physics simulations. Furthermore some handrails to help prevent situations could be put in place for instance pausing the simulation when switching structures or warning the user when loading large structures. Another solution to the lagginess when loading large files is to instead of attempting to load all files in one frame instead do each file iteratively across multiple frames as to reduce extreme temporary freezes.

Over all the beta testing showed that my main objectives have been met but highlighted that robustness is key in a complete application. I should work on stability and hand rails before being put in a classroom setting.

Improvements

I have identified 5 main Improvements

Stability/performance

A massive pitfall to my application is its tenacity to slow down to a halt when trying to perform large simulations. Complicated Simulations should not prevent the user from interacting with the simulation to reduce the workload or iterate manually. This could be solved in two ways: putting the physics on a separate thread to the rendering and secondly parallelising the entire physics simulation allowing it to improve in performance by 2x or more.

Usability

Lots of feedback I gathered from users during development was that the mandatory keybinds should be more obvious to the user. This was a feature early on in development

but was caught in the crossfire of changing over to use ImGui over my own in house GUI system. This can be done by putting a piece of text next to the cursor reading out what key the user should press and why.

Database/Assignment improvement

When Designing the central database I added lots of entries in the assignment system that were never used in the final assignment system and so it is only natural that I would improve upon the current assignment and database system. Currently only adding more assignments is possible, not grading and returning them. Also Administration is not currently possible within the program so adding more users is impossible without directly accessing the database which is not user friendly.

Tearing/Stick Improvement

Currently the constraint system is extremely simple. And linear in real life springs deform as the extent and evaluations even snap. A future improvement would allow for this. A system where springs when stretched beyond their limit will not rebound back and if stretched enough will snap.

Notes.

A greatly missed feature would be a note system where the user could add Notes to the simulation in ImGui windows. These notes would be saved to JSON or postgres so they can be loaded by anyone. This would allow for better explanations of the users creation